

Mask-Aware Shapley Attribution

1 Shapley Values for Cooperative Games

Consider a **coalitional game** with player set N where any subset (**coalition**) $S \subseteq N$ can form and achieve value given by a **characteristic function** (also called a **value function**) $v : 2^N \rightarrow \mathbb{R}$ with $v(\emptyset) = 0$. In the machine learning context we will refer to v as the **value function**. The problem is to find a fair **allocation** $\phi_i(v)$ for each player $i \in N$ that distributes $v(N)$ according to agreed fairness principles.

Four axioms define any fair **allocation mechanism**:

1. **Efficiency:** The axiom requires $\sum_{i \in N} \phi_i(v) = v(N)$. It ensures the full value of the player set is distributed.
2. **Symmetry:** The axiom states that if $v(S \cup \{i\}) = v(S \cup \{j\})$ for all S not containing i or j , then $\phi_i(v) = \phi_j(v)$. It enforces equal treatment of players with identical marginal contributions.
3. **Dummy Player:** The axiom states that if $v(S \cup \{i\}) = v(S)$ for all S not containing i , then $\phi_i(v) = 0$. It recognises that players who never contribute value receive none of the payoff.
4. **Linearity:** The axiom requires $\phi_i(av + bw) = a\phi_i(v) + b\phi_i(w)$ for any value functions v, w and scalars $a, b \in \mathbb{R}$. It ensures additivity across games and positive homogeneity with respect to scaling.

These four axioms uniquely determine the allocation. The **Shapley value** for player i is given by:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)] \quad (1)$$

Derivation of the weight $\frac{|S|!(|N| - |S| - 1)!}{|N|!}$: Consider all $|N|!$ possible arrival orders (**permutations**) of the N players. For player i to join exactly the coalition S , the players in S must arrive before i , and the remaining $|N| - |S| - 1$ players must arrive after i . There are $|S|!$ ways to order the players arriving before i , and $(|N| - |S| - 1)!$ ways to order those arriving after i . Thus, the number of permutations where i joins coalition S is $|S|!(|N| - |S| - 1)!$, and the

probability is $\frac{|S|!(|N|-|S|-1)!}{|N|!}$. Therefore the Shapley value equals the expected marginal contribution under a uniform random arrival order.

The term $v(S \cup \{i\}) - v(S)$ is player i 's **marginal contribution** when joining coalition S .

Equivalently, averaging over all **permutations** $\pi \in \Pi(N)$, where $\Pi(N)$ denotes the set of all permutations of N :

$$\phi_i(v) = \frac{1}{|N|!} \sum_{\pi \in \Pi(N)} [v(S_\pi^i \cup \{i\}) - v(S_\pi^i)] \quad (2)$$

where S_π^i is the set of players appearing before i in permutation π .

2 Machine Learning Feature Attribution as Shapley Values

2.1 Formal Setup

We now set up the general SHAP framework for feature attribution. Consider a **predictive model** $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and an input $x \in \mathbb{R}^n$ with prediction $f(x)$. The goal is to quantify how much each feature in x contributes to this prediction. We formulate this task as a Shapley value computation by treating each **original feature as a player** in a cooperative game. We index the features by the player set $N = \{1, \dots, n\}$, with feature i corresponding to coordinate x_i .

To connect the model f with coalitional games we introduce the **masked predictor** $f(x; S)$. For any subset (**coalition**) $S \subseteq N$ we interpret S as the set of features that are treated as known, and we define $f(x; S)$ to be the model prediction when only the coordinates in S are available and the remaining coordinates $N \setminus S$ are treated as missing. Notice that we have not yet specified how to compute $f(x; S)$ in practice. This is a **major modeling choice** when computing Shapley values, and we will discuss it in detail in the next section. The special case $f(x; \emptyset)$ is the prediction when no feature information is used, and we refer to this quantity as the **baseline prediction**. We therefore set

$$b = f(x; \emptyset).$$

Given this masked predictor we define the **value function** $v : 2^N \rightarrow \mathbb{R}$ by

$$v(S) = f(x; S) - f(x; \emptyset)$$

for every coalition $S \subseteq N$. By construction, $v(\emptyset) = 0$ and $v(N) = f(x) - f(x; \emptyset)$, so the total game value equals the difference between the actual prediction and the baseline prediction.

A **Shapley explanation** of f at input x is a vector of feature attributions $\phi(x) = (\phi_1(x), \dots, \phi_n(x))$ associated with the game (N, v) . The SHAP framework enforces the **efficiency** condition

$$v(N) = f(x) - f(x; \emptyset) = \sum_{i=1}^n \phi_i(x),$$

so that the total attribution exactly matches the game value $v(N)$. Each attribution $\phi_i(x)$ is defined to be the Shapley value of feature i in the cooperative game (N, v) . Formally, writing $n = |N|$, the Shapley value for feature i is

$$\phi_i(x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (n - |S| - 1)!}{n!} [v(S \cup \{i\}) - v(S)].$$

This expression averages feature i 's **marginal contribution** $v(S \cup \{i\}) - v(S)$ over all coalitions S that do not contain i , with weights corresponding to the fraction of permutations in which S precedes i . Equivalently, let $\Pi(N)$ denote the set of all permutations of N and let S_π^i be the set of features that appear before i in permutation π . Then we can write

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} [v(S_\pi^i \cup \{i\}) - v(S_\pi^i)]$$

and, using the definition of v , this becomes

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} [f(x; S_\pi^i \cup \{i\}) - f(x; S_\pi^i)].$$

Thus, in the SHAP setting the Shapley values admit two complementary interpretations. First, they **allocate the prediction above the baseline** $f(x; \emptyset)$ so that the attributions sum to $f(x) - f(x; \emptyset)$. Second, each $\phi_i(x)$ equals the **expected marginal change in prediction** when feature i is added to the set of previously included features in a uniformly random ordering.

2.2 Intuition: Toy House Price Example

Suppose we observe five houses, each described by three features—lot size (acres), total square footage, and number of rooms—and a sale price y in dollars, as shown in Table 1.

House	Acres	Square feet	Rooms	Price (\$)
1	0.10	1200	3	220,000
2	0.15	1400	3	260,000
3	0.20	1600	4	300,000
4	0.25	1800	4	340,000
5	0.30	2000	5	380,000

Table 1: Toy dataset of five houses with simple features and sale prices.

Let $x \in \mathbb{R}^3$ collect the three features for a house and let $f(x)$ be a trained regression model's predicted sale price. For a particular house x , for example $x_5 = (0.30, 2000, 5)$, the feature attribution problem is to determine how each coordinate of x (acres, square footage, rooms) contributes to the prediction

$f(x)$. Assume for concreteness that the model outputs $f(x_5) = 360,000$ dollars for house 5.

In the notation of the formal setup, we consider the extended prediction operator $f(x_5; S)$ for coalitions $S \subseteq N = \{1, 2, 3\}$, where the three features correspond to acres, square footage, and rooms. We define the **baseline prediction** to be $f(x_5; \emptyset)$, which in this example we take to be the average sale price across the five houses. If the average price is 300,000 dollars, then $f(x_5) - f(x_5; \emptyset) = 60,000$ dollars. This quantity is the total value $v(N)$ that must be allocated across the three features by the Shapley attributions. The feature attributions decompose the 60,000-dollar improvement above the baseline. Equivalently, we can interpret each $\phi_i(x_5)$ as the expected marginal change in the prediction when feature i is added to a coalition of previously included features in a uniformly random order. Concretely, we start from the baseline prediction $f(x_5; \emptyset)$, choose a random ordering of the three features, and then add them one at a time, recording the increment $f(x_5; S \cup \{i\}) - f(x_5; S)$ each time feature i is added to a set S of features that appear earlier in the ordering. The Shapley value $\phi_i(x_5)$ is the average of this marginal change across all six possible orderings.

To make this explicit, label acres, square footage, and rooms as features 1, 2, 3 so that $N = \{1, 2, 3\}$. For feature 1, the six permutations of N are

$$(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1).$$

In each permutation π , let S_π^1 be the set of features that appear before 1. The marginal contribution of feature 1 in permutation π is

$$\Delta_1^\pi = f(x_5; S_\pi^1 \cup \{1\}) - f(x_5; S_\pi^1).$$

For example, in permutation $(1, 2, 3)$ we have $S_{(1,2,3)}^1 = \emptyset$ and

$$\Delta_1^{(1,2,3)} = f(x_5; \{1\}) - f(x_5; \emptyset),$$

while in permutation $(2, 1, 3)$ we have $S_{(2,1,3)}^1 = \{2\}$ and

$$\Delta_1^{(2,1,3)} = f(x_5; \{1, 2\}) - f(x_5; \{2\}).$$

The Shapley value for feature 1 is then

$$\phi_1(x_5) = \frac{1}{6} \sum_{\pi \in \Pi(\{1,2,3\})} \Delta_1^\pi.$$

Analogous expressions hold for features 2 and 3, obtained by averaging their marginal contributions over the same six permutations.

3 Defining the Masked Predictor

In the previous section, we defined the value function $v(S) = f(x; S) - f(x; \emptyset)$ and introduced the **baseline prediction** $f(x; \emptyset)$. We did not yet specify how

the **masked predictions** $f(x; S)$ are computed. The predictive model f is usually defined only on full feature vectors x , so we must decide how to evaluate the **masked predictor** $f(x; S)$ when only a subset of features is treated as known.

Shapley explanations depend critically on how we define the masked predictor because it is central to the value function:

1. **Baseline prediction:** For a fixed input x , the baseline prediction $f(x; \emptyset)$ determines the total quantity $f(x) - f(x; \emptyset)$ that the Shapley values $\phi_i(x)$ must allocate. The value function $v(S) = f(x; S) - f(x; \emptyset)$ depends directly on the chosen baseline.
2. **Masked predictor semantics:** The masked predictions $f(x; S)$ specify how this total value is distributed across coalitions S . Different definitions of $f(x; S)$ induce different value functions $v(S)$ and thus different Shapley explanations, even when the underlying model f and the input x are fixed.
3. **Faithfulness to the original model:** The Shapley game is built on the masked predictor $S \mapsto f(x; S)$ rather than directly on the original predictor $x \mapsto f(x)$. If $f(x; S)$ is defined using unrealistic hybrids or an inappropriate background distribution, the resulting value function $v(S)$ can behave very differently from f on typical inputs, and the Shapley attributions may fail to faithfully reflect how f actually uses its features.

Several standard choices exist for defining $f(x; S)$. Intuitively, the **masked predictor** $f(x; S)$ represents the prediction when features in S are treated as known and features in $N \setminus S$ are treated as unknown. We evaluate $f(x; S)$ by applying f to a full feature vector in which coordinates in S are fixed to x_S and coordinates in $N \setminus S$ are filled in according to a chosen rule. Several standard choices for this rule appear in the SHAP literature:

1. **Fixed baseline substitution:** Choose a baseline input $x^{(0)} \in \mathbb{R}^n$ (for example, a zero vector or a reference input) and define a hybrid input that uses x_S on coordinates in S and $x_{N \setminus S}^{(0)}$ on coordinates in $N \setminus S$. The masked predictor is $f(x; S) = f(x_S, x_{N \setminus S}^{(0)})$. Unknown features are always replaced with fixed baseline values, which is simple but can evaluate f on out-of-distribution inputs. Formally, for each coalition $S \subseteq N$ we define a hybrid input $x^{(S)} \in \mathbb{R}^n$ by

$$x_i^{(S)} = \begin{cases} x_i, & i \in S, \\ x_i^{(0)}, & i \notin S, \end{cases}$$

so that $x^{(S)} = (x_S, x_{N \setminus S}^{(0)})$ and

$$f(x; S) := f(x^{(S)}).$$

In words, $x^{(S)}$ agrees with the original input x on coordinates in S and with the baseline input $x^{(0)}$ on coordinates in $N \setminus S$.

2. **Mean imputation:** Let $\bar{x}_{N \setminus S} = \mathbb{E}[X_{N \setminus S}]$ denote the mean of the unknown features under the training distribution. The masked predictor is $f(x; S) = f(x_S, \bar{x}_{N \setminus S})$. Unknown features are replaced with their mean training values, which approximates averaging over the marginal distribution using only its first moment.
3. **Marginal expectation (interventional semantics):** Let $P(X)$ denote the joint training distribution over features. The masked predictor is defined by the *marginal expectation*

$$f(x; S) = \mathbb{E}_{X_{N \setminus S} \sim P(X_{N \setminus S})} [f(x_S, X_{N \setminus S})].$$

Unknown features are sampled from their marginal distribution, so conditioning on x_S does not affect their distribution. In practice, this expectation is approximated using samples from the empirical training distribution.

4. **Conditional expectation (conditional semantics):** Instead of breaking feature dependencies, we can preserve them by averaging over the conditional distribution of the unknown features given the known ones. The masked predictor is

$$f(x; S) = \mathbb{E}_{X_{N \setminus S} \sim P(X_{N \setminus S} | X_S = x_S)} [f(x_S, X_{N \setminus S})].$$

Unknown features follow their conditional distribution given $X_S = x_S$, which properly accounts for feature correlations but requires modeling $P(X_{N \setminus S} | X_S)$ in high dimensions.

A mask-aware approach—detailed in the closing section—replaces these ad hoc choices with a learned semantics of missingness. By exposing the model to random masks during training, we define $f(x; S)$ directly through a mask channel rather than through hybrids built from external backgrounds. This keeps $f(x; S)$ on-manifold, makes $f(x; \emptyset)$ an in-model baseline, and gives access to the conditional behaviour the Shapley value requires without separately estimating $P(X_{N \setminus S} | X_S)$.

From a prediction point of view, the **conditional expectation** definition yields the most faithful notion of prediction when only a subset of features is known. Let X denote the random input vector with distribution $P(X)$ and consider predicting the scalar response $f(X)$ under squared error loss when only X_S is observed. The prediction that minimises the expected squared error given $X_S = x_S$ is

$$g_S(x_S) = \mathbb{E}[f(X) | X_S = x_S],$$

which is exactly the conditional masked predictor obtained by averaging $f(x_S, X_{N \setminus S})$ over the conditional distribution of the unknown coordinates $X_{N \setminus S}$ given $X_S = x_S$. In words, conditional expectation computes the best mean-squared-error prediction for $f(X)$ using all information in the observed features X_S while sampling the unknown features in a way that respects their empirical dependencies with X_S . By contrast, marginal expectation ignores these dependencies

by treating $X_{N \setminus S}$ as independent of X_S , and fixed-baseline or mean imputation correspond to degenerate choices of $P(X_{N \setminus S} \mid X_S)$ that are no longer optimal given X_S in this squared-error sense. Thus, when the goal is to explain predictions under the observed data distribution and to treat known and unknown coordinates coherently, conditional expectation provides a principled notion of “best prediction given x_S ” for defining the masked predictor.

This perspective also clarifies why simple **zero** or **mean** baselines can produce unfaithful Shapley games. A zero baseline sets $x^{(0)}$ to the all-zero vector and evaluates hybrids $f(x_S, x_{N \setminus S}^{(0)})$ that may lie far outside the support of the training distribution, so the value function $v(S)$ reflects model behaviour on highly unrealistic inputs rather than on typical data. Similarly, a mean baseline or mean imputation uses a single fixed vector or the marginal mean $\bar{x}_{N \setminus S}$ for all coalitions S , which discards the dependence between X_S and $X_{N \setminus S}$ and does not approximate the conditional expectation $\mathbb{E}[f(X) \mid X_S = x_S]$. In both cases, the masked predictor $S \mapsto f(x; S)$ is anchored to off-manifold or weakly informative reference points, so the induced value function $v(S)$ can deviate substantially from how f behaves on realistic inputs. As a result, Shapley attributions built on zero or mean baselines may emphasise artefacts of the chosen reference rather than faithfully capturing how the original model uses its features across the data distribution.

4 Computational Challenges and Practical Approximations

In the SHAP setting we work with the permutation-based definition of the Shapley value, since it matches how most practical algorithms operate (they follow a random feature ordering and add features one at a time). For a fixed input x and feature i , this can be written as

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} \left[f(x; S_\pi^i \cup \{i\}) - f(x; S_\pi^i) \right],$$

where $\Pi(N)$ is the set of all permutations of $N = \{1, \dots, n\}$ and S_π^i is the set of features that appear before i under permutation π . This “add features along a random ordering” view makes clear what must be computed and where the cost arises.

4.1 Summation Over Permutations

The first bottleneck is the summation over permutations. The exact Shapley formula involves $n!$ permutations for each feature i , and for each permutation π we must evaluate the difference

$$f(x; S_\pi^i \cup \{i\}) - f(x; S_\pi^i).$$

This factorial dependence on n quickly becomes infeasible as the number of features grows.

In practice, permutation-based SHAP implementations approximate this sum by drawing a limited number of random permutations and averaging the corresponding marginal contributions along each ordering. For deep networks, DeepSHAP methods approximate the same permutation sum more aggressively by exploiting the compositional structure of the network: they use DeepLIFT-style forward and backward rules to collapse the contribution of all permutations into a single forward and backward pass per background example, yielding an efficient approximation to the Shapley values without explicitly enumerating permutations.

4.2 Masked Predictor Evaluation

The second bottleneck concerns the cost of evaluating the masked predictor $f(x; S)$ itself. Each term in the permutation sum requires computing

$$f(x; S_\pi^i) \quad \text{and} \quad f(x; S_\pi^i \cup \{i\})$$

for some coalition S_π^i . Let C_{mask} denote the cost of a single masked evaluation; then

$$\text{total cost} = (\text{number of masked evaluations}) \times C_{\text{mask}}.$$

The value of C_{mask} depends entirely on how $f(x; S)$ is defined:

- **Fixed baseline substitution / mean imputation.** Missing features are replaced by a fixed reference input (zeros, feature-wise means, or a hand-crafted baseline), so $f(x; S)$ is obtained with a single forward pass of the original model f . In this case C_{mask} is essentially the cost of ordinary prediction.
- **Marginal (interventional) expectation.** Missing features are sampled independently from a background dataset drawn from the training distribution. In practice one chooses a finite set of background points $\{x^{(b)}\}_{b=1}^B$ and approximates

$$f(x; S) \approx \frac{1}{B} \sum_{b=1}^B f(x_S, x_{N \setminus S}^{(b)}),$$

which requires B forward passes of f for each coalition S . Here C_{mask} scales linearly with the number of background samples B .

- **Conditional expectation.** The conditional semantics

$$f(x; S) = \mathbb{E}[f(x_S, X_{N \setminus S}) \mid X_S = x_S]$$

require access to the conditional distribution $P(X_{N \setminus S} \mid X_S = x_S)$. Estimating this conditional typically entails training an additional high-dimensional imputation or generative model for X and then drawing conditional samples for each coalition S , which substantially increases C_{mask}

beyond the cost of running f itself. Because of this extra modelling burden and computational overhead, such conditional estimators are rarely used in mainstream SHAP workflows, which overwhelmingly adopt the simpler marginal or baseline-based semantics described above.

Putting these pieces together, the de facto standard in modern deep-learning applications is a combination of *marginal* masking semantics with a DeepSHAP-style approximation to the permutation sum. Concretely, one selects a background dataset $\{x^{(b)}\}_{b=1}^B$ drawn from the training distribution and approximates each masked evaluation by

$$f(x; S) \approx \frac{1}{B} \sum_{b=1}^B f(x_S, x_{N \setminus S}^{(b)}).$$

Let C_f denote the cost of a single forward and backward pass of the network on one input. DeepSHAP collapses the $n!$ -term permutation sum into a single forward-backward pass *per background point* by using DeepLIFT-style propagation rules, so that computing a complete set of Shapley attributions for one input x costs on the order of $B \cdot C_f$. In practice B is typically chosen between ten and a few hundred, so the overall computational budget is dominated by B evaluations of the base network.

This status quo explains the tension highlighted above. On the one hand, the Shapley value is formally defined in terms of the conditional masked predictor $S \mapsto \mathbb{E}[f(X) \mid X_S = x_S]$, which provides the Bayes-optimal notion of “prediction under partial information”. On the other hand, conditional masking is rarely implemented in practice because it requires an additional high-dimensional generative model and greatly increases C_{mask} . As a result, most practical SHAP pipelines adopt marginal or baseline-substitution semantics paired with DeepSHAP, trading off the faithfulness of the underlying value function $v(S)$ in exchange for tractability.

5 Input Mask-Aware Models for Shapley Attribution

The analysis in the previous sections singles out the **conditional masked predictor**

$$\mathbb{E}[f(X) \mid X_S = x_S]$$

as the most faithful way to define the value function $v(S)$ when only a coalition S of features is treated as known. In practice, however, this requires access to the conditional distribution $P(X_{N \setminus S} \mid X_S)$. Our goal in this section is to approximate these conditional masked predictions with a single model that is trained from the beginning to handle missing features.

The key idea is a *mask-aware* approach: instead of defining the masked predictor $S \mapsto f(x; S)$ externally via baselines or background samples, we train

a predictive model that takes as input both the feature vector and an explicit mask that indicates which coordinates are observed. This model is exposed during training to random coalitions of observed features, so that it learns to make predictions from partial information. Once such a model is trained, we use it as the masked predictor inside the Shapley game and compute feature attributions by sampling permutations as in the original Shapley definition.

5.1 Training with Feature Masks

We introduce an *input mask-aware* predictor

$$\tilde{f} : \mathbb{R}^n \times \{0, 1\}^n \rightarrow \mathbb{R}$$

that acts on pairs consisting of a feature vector and a binary mask. We write $z = (x, m)$ for such a pair, where $m_i = 1$ indicates that feature i is observed and $m_i = 0$ indicates that feature i is masked. During training, each example (x, y) from the data distribution is paired with a randomly sampled mask $m \in \{0, 1\}^n$. We form a masked input together with its mask and train $\tilde{f}$ to predict y .

To make the distinction between “unknown” features and features that are genuinely equal to zero explicit, we use a masking scheme with an additional mask channel. Concretely, we zero out masked coordinates *and* append a binary indicator for whether each feature is used, bundling the result into a single input vector:

$$z = (x \odot m, m),$$

where $\odot$ denotes coordinatewise multiplication and z can be viewed as a $2 \times n$ matrix or a $2n$ -dimensional vector with one row for masked feature values and one row for mask bits (1 if the feature is used, 0 if it is unknown). For masked features we therefore pass $x_i \odot m_i = 0$ together with $m_i = 0$, so the network can learn that these zeros correspond to missing information rather than actual zero-valued features. Writing M for a random mask drawn from some distribution over $\{0, 1\}^n$ that includes the full-information mask $M = \mathbf{1}$, we minimise an expected loss of the form

$$\mathbb{E} \left[\ell(\tilde{f}(X \odot M, M), Y) \right]$$

over the parameters of $\tilde{f}$. Intuitively, this training procedure encourages the model to make coherent predictions from arbitrary coalitions S of observed features. After training, we define the **mask-aware masked predictor** by

$$f_{\text{mask}}(x; S) := \tilde{f}(x \odot \mathbf{1}_S, \mathbf{1}_S),$$

where $\mathbf{1}_S \in \{0, 1\}^n$ is the indicator of coalition S . Because masked inputs of this form appear during training, the evaluations $S \mapsto f_{\text{mask}}(x; S)$ remain on-manifold with respect to the training distribution of masked examples. Heuristically, $f_{\text{mask}}(x; S)$ acts as a data-driven approximation to the conditional masked predictor $g_S(x_S)$ without requiring an explicit model for $P(X_{N \setminus S} \mid X_S)$. This mask-aware construction directly addresses the three issues highlighted above:

1. **Baseline prediction:** The baseline that anchors the game is no longer imposed by an arbitrary reference vector such as a zero or mean baseline; instead, $f_{\text{mask}}(x; \emptyset)$ is learned in-model from masked training examples.
2. **Masked predictor semantics:** The semantics of the masked predictor are implemented by an explicit mask channel that tells the network which coordinates are known and which are unknown, rather than by feeding hybrids that splice together x and a baseline input. In particular, we avoid “half baseline” inputs of the form $(x_S, x_{N \setminus S}^{(0)})$ or $(x_S, \bar{x}_{N \setminus S})$ that can place the model far off the data manifold.
3. **Faithfulness to the original model:** Every masked input used in the Shapley game has the same structure $(x \odot \mathbf{1}_S, \mathbf{1}_S)$ that the predictor was exposed to during training, which improves the faithfulness of $v_{\text{mask}}(S)$ to how the underlying model uses its features on realistic inputs.

5.2 Shapley Values with a Mask-Aware Masked Predictor

Given $f_{\text{mask}}(x; S)$, we define the associated value function

$$v_{\text{mask}}(S) = f_{\text{mask}}(x; S) - f_{\text{mask}}(x; \emptyset)$$

for all coalitions $S \subseteq N$. By construction, $v_{\text{mask}}(\emptyset) = 0$, and

$$v_{\text{mask}}(N) = f_{\text{mask}}(x; N) - f_{\text{mask}}(x; \emptyset) = f(x) - f_{\text{mask}}(x; \emptyset),$$

so that Shapley attributions for the game (N, v_{mask}) decompose the prediction above the mask-aware baseline $f_{\text{mask}}(x; \emptyset)$.

We then compute Shapley values in the usual permutation-based way, simply replacing the original masked predictor $f(x; S)$ with $f_{\text{mask}}(x; S)$. For feature i we have

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} \left[f_{\text{mask}}(x; S_{\pi}^i \cup \{i\}) - f_{\text{mask}}(x; S_{\pi}^i) \right],$$

where S_{π}^i is the set of features that appear before i in permutation π . As before, this can be interpreted as the expected marginal change in the prediction when feature i is added to a coalition of previously included features in a uniformly random ordering, but now the marginal contributions are computed with respect to the mask-aware masked predictor f_{mask} .

In practice, we approximate the average over permutations by Monte Carlo sampling. Let P denote the number of sampled permutations. For each permutation π we traverse the features once, maintain the growing coalition $S_{\pi}^0 \subset S_{\pi}^1 \subset \dots \subset S_{\pi}^n$, and record the corresponding increments

$$f_{\text{mask}}(x; S_{\pi}^k \cup \{\pi_{k+1}\}) - f_{\text{mask}}(x; S_{\pi}^k), \quad k = 0, \dots, n-1.$$

Averaging these increments over permutations yields an estimate of the Shapley values $\phi_i(x)$.

Let C_f^{mask} denote the cost of a single forward pass of the mask-aware predictor $\tilde{f}$ on one masked input. The Monte Carlo estimator described above requires on the order of $P \cdot n$ masked evaluations per input, so the total cost of computing all feature attributions for a single x is

$$\text{cost} \approx P \cdot n \cdot C_f^{\text{mask}}.$$

Each masked evaluation is a single forward pass of the same network used for ordinary prediction, and no background dataset or separate conditional sampler is required.

6 Discussion and Notes

Permutation cost and scalability. Although the mask-aware approach changes how the masked predictor $S \mapsto f(x; S)$ is defined, it does not remove the combinatorial structure of the Shapley value itself. Exact evaluation still involves $n!$ permutations per feature, which is infeasible even for moderate n . Our method therefore relies on the same Monte Carlo approximation over permutations as classical Shapley sampling schemes: with P sampled permutations, the cost per input is on the order of $P \cdot n$ forward passes of the mask-aware predictor. This is tractable for moderate n and moderate P , but the permutation sampling step remains a dominant bottleneck for very high-dimensional inputs.

Relation to marginal SHAP and DeepSHAP. Mainstream SHAP implementations for deep networks typically adopt marginal or baseline-based masking semantics and accelerate the permutation sum using DeepSHAP (or more generally DeepLIFT-style) propagation rules. As discussed in Section ??, this yields a cost on the order of $B \cdot C_f$ per input, where B is the number of background points and C_f is the cost of one forward and backward pass of the original network. Our proposal moves in the opposite direction. We change the masked predictor to a mask-aware model that approximates the conditional masked predictor $g_S(x_S)$, and we then apply the original permutation-based Shapley definition to this learned value function. Because the semantics of missingness are now encoded inside $\tilde{f}$, the DeepLIFT-style collapse of the permutation sum does not apply in the same way, and we pay the $P \cdot n$ factor from permutation sampling. The intended benefit is that v_{mask} is closer to the conditional Shapley game than the marginal or fixed-baseline alternatives that DeepSHAP typically uses.

Design choices and variants. Several components of the mask-aware construction can be varied without changing the overall framework. These include the distribution over masks used during training (for example, focusing on coalitions of a given size, or biasing towards small or large $|S|$), the way in which masks are injected into the network (zeroing inputs, using learned mask

embeddings, or both), and the number P of permutations used during attribution. In our experiments we adopt simple, architecture-agnostic choices for these components and focus on the central question: whether replacing marginal or baseline-based masking with a learned mask-aware predictor leads to Shapley attributions that better approximate the conditional game (N, v) on both synthetic and real datasets.